

# Metodi di ottimizzazione per la classificazione della malattia cardiaca mediante regressione logistica - Cleveland Heart Disease Case Study

Rosachiara D'Onghia

## Abstract

Le malattie cardiovascolari rappresentano una delle principali cause di mortalità a livello globale. In questo contesto, le tecniche di Machine Learning offrono un supporto alla diagnosi clinica, permettendo di estrarre pattern predittivi dai dati. Il presente lavoro affronta lo studio, dal punto di vista teorico e computazionale, dei principali algoritmi di ottimizzazione per la classificazione binaria su dati clinici.

A partire dal dataset *Cleveland Heart Disease (UCI)*, viene inizialmente condotta un'analisi esplorativa dei dati mirata alla gestione dei valori mancanti e alla trasformazione delle feature tramite tecniche di imputazione, standardizzazione e one-hot encoding.

Il task di classificazione binaria viene formalizzato attraverso un modello di Regressione Logistica, implementato da zero, il cui funzionale obiettivo da minimizzare è formulato matematicamente come un problema di *finite-sum minimization*, composto da un termine di rischio empirico basato sulla *Log-Loss* e da un termine di regolarizzazione  $L_2$ . Questa struttura garantisce un funzionale convesso e sufficientemente regolare, fornendo un contesto appropriato per l'applicazione e l'analisi teorica dei metodi di discesa.

È stato eseguito un confronto approfondito tra diversi algoritmi di ottimizzazione non vincolata (Discesa del Gradiente, Metodo di Newton, Newton-CG, BFGS e L-BFGS).

Questi metodi sono stati integrati con strategie line search basate sulle condizioni di Armijo (Backtracking) e Wolfe per la determinazione dinamica dello stepsize.

L'ottimizzazione dell'iperparametro di regolarizzazione  $\lambda$  è stata condotta tramite Grid Search e Stratified  $K$ -Fold Cross-Validation ( $K = 5$ ) sul training set, individuando in  $\lambda = 1$  la configurazione ottimale per minimizzare la Log-Loss.

Il modello è stato valutato su un test set di 61 pazienti, raggiungendo un'accuratezza dell'87%, un'Area Sotto la Curva ROC (AUC) pari a 0.958 e una Sensibilità (Recall) del 93% sui casi patologici, limitando i Falsi Negativi a soli 2 casi. L'analisi dei coefficienti appresi ha garantito l'interpretabilità del classificatore, confermando l'impatto clinico di variabili chiave come il sesso, il numero di vasi alla fluoroscopia, il tipo di dolore toracico e la presenza di un difetto di perfusione reversibile.

# 1 Heart Disease Dataset

Per questo progetto è stato utilizzato il dataset **Heart Disease**, proveniente dall'UCI Machine Learning Repository [1]. Sebbene il repository originale includa dati raccolti da quattro diverse istituzioni cliniche relative a pazienti di Cleveland, Switzerland, Long Beach VA e Statlog, questo studio si basa esclusivamente sul **Dataset di Cleveland**, il quale rappresenta lo standard storico utilizzato dalla comunità scientifica per gli esperimenti di Machine Learning.

Il dataset è stato inoltre sottoposto a rigorosi processi di anonimizzazione: per rispettare le normative sulla privacy medica, i nomi e i numeri di previdenza sociale dei pazienti sono stati completamente rimossi e sostituiti con valori fittizi.

## 1.1 Descrizione e struttura del Dataset

Il dataset originale raccoglieva 76 attributi clinici per paziente. La variante (Cleveland) si compone di un totale di 303 osservazioni e 13 variabili predittive (feature) attributi clinici/demografici (*età*,  *Sesso*, tipo di dolore toracico *cp*, pressione sistolica *trestbps*, colesterolo *chol*, glicemia a digiuno *lbs*, ECG *restecg*, frequenza cardiaca massima *thalach*, angina da sforzo *exang*, depressione ST *oldpeak*, pendenza *slope*, numero di vasi *ca*, test alla talassemia *thal*) ed 1 variabile di output (target).

Le 13 variabili coprono diverse aree dell'indagine clinica, dai semplici dati demografici fino agli esami strumentali a riposo e sotto sforzo, e i test diagnostici avanzati.

Nella Tabella 1 sono descritte in dettaglio tutte le variabili presenti nel dataset.

| Variabile       | Ruolo   | Tipo       | Descrizione e Valori                                                                                                                                                                                                                                         |
|-----------------|---------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>age</b>      | Feature | Numerica   | Età del paziente in anni.                                                                                                                                                                                                                                    |
| <b>sex</b>      | Feature | Binaria    | Sesso del paziente: <b>1 = Maschio, 0 = Femmina</b> .                                                                                                                                                                                                        |
| <b>cp</b>       | Feature | Categorica | Tipo di dolore toracico riferito. <b>1 = Angina tipica</b> (opprimente, da sforzo/stress); <b>2 = Angina atipica</b> ; <b>3 = Dolore non anginoso</b> (non di origine cardiaca); <b>4 = Asintomatico</b> .                                                   |
| <b>trestbps</b> | Feature | Numerica   | Pressione sanguigna a riposo (in mm Hg al momento del ricovero in ospedale). Valori elevati indicano ipertensione.                                                                                                                                           |
| <b>chol</b>     | Feature | Numerica   | Colesterolo sierico in mg/dl. Livelli elevati favoriscono la formazione di placche che ostruiscono le arterie coronarie.                                                                                                                                     |
| <b>fbs</b>      | Feature | Binaria    | Glicemia a digiuno > 120 mg/dl: <b>1 = Vero, 0 = Falso</b> . È un forte indicatore di pre-diabete o diabete, che aumenta il rischio cardiovascolare.                                                                                                         |
| <b>restecg</b>  | Feature | Categorica | Esito dell'elettrocardiogramma a riposo. <b>0 = Normale</b> ; <b>1 = Anomalia del tratto ST-T</b> (segno di ischemia o infarto); <b>2 = Iper-trofia ventricolare sinistra probabile o certa secondo i criteri di Estes</b> (spesso causata da ipertensione). |
| <b>thalach</b>  | Feature | Numerica   | Frequenza cardiaca massima raggiunta (bpm)                                                                                                                                                                                                                   |
| <b>exang</b>    | Feature | Binaria    | Angina (dolore toracico) indotta dall'esercizio fisico: <b>1 = Sì, 0 = No</b> . È uno dei predittori più forti di ostruzione coronarica.                                                                                                                     |
| <b>oldpeak</b>  | Feature | Numerica   | Depressione del tratto ST sull'ECG (in mm) indotta dall'esercizio rispetto al riposo. Valori alti indicano ischemia grave.                                                                                                                                   |
| <b>slope</b>    | Feature | Categorica | Pendenza del segmento ST di picco durante l'esercizio. <b>1 = In salita</b> (Upsloping); <b>2 = Pianeggiante</b> (Flat, indica ischemia da sforzo severa); <b>3 = In discesa</b> (Downsloping, indica malattia diffusa).                                     |
| <b>ca</b>       | Feature | Numerica   | Numero di vasi sanguigni coronarici principali (da <b>0 a 3</b> ) colorati e resi visibili tramite fluoroscopia a mezzo di contrasto. I vasi ostruiti da placche non si colorano.                                                                            |
| <b>thal</b>     | Feature | Categorica | Esito della scintigrafia al tallio (perfusione del sangue). <b>3 = Normale</b> ; <b>6 = Difetto fisso</b> (tessuto morto/cicatrizzato da infarto passato); <b>7 = Difetto reversibile</b> (tessuto vivo ma in ischemia, ad alto rischio).                    |
| <b>target</b>   | Target  | Binaria    | Diagnosi di cardiopatia( stato della malattia angiografica. <b>0 = Assenza di malattia</b> (restringimento vasi < 50%); <b>1 = Presenza di malattia</b> (raggruppa i valori originali da 1 a 4, che indicavano gravità crescenti).                           |

Tabella 1: Variabili del dataset UCI Heart Disease

## 1.2 Analisi esplorativa e preprocessing

L'obiettivo del modello predittivo è la diagnosi della patologia cardiaca, configurando il task come un problema di classificazione binaria. Nel dataset, il target *num* indica la diagnosi angiografica della malattia cardiaca, si presenta come un valore intero compreso tra 0 e 4:

- 0: assenza di malattia (restringimento coronarico inferiore al 50%).
- 1, 2, 3, 4: presenza della patologia con gradi crescenti di gravità (restringimento superiore al 50% in uno o più vasi).

In linea con gli esperimenti standard condotti sul database di Cleveland, il problema è stato ricondotto a un problema di classificazione binaria.

La variabile target è stata trasformata in un vettore binario:

- classe 0: pazienti sani, per i quali si conferma un restringimento dei vasi < 50% (mantenendo il valore diagnostico originale 0);
- classe 1: pazienti affetti da patologia cardiaca, ottenuta raggruppando sotto un'unica etichetta i valori diagnostici originali compresi tra 1 e 4.

Il dataset è formalizzato come un insieme di coppie di samples:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$$

dove:

- $N = 303$  è il numero totale di pazienti;
- $x_i$  è il vettore delle feature associate al paziente  $i$ ;
- $y_i$  è il target associato al paziente  $i$ .

Per garantire una valutazione robusta delle prestazioni, il dataset è stato inizialmente suddiviso in un insieme di addestramento  $\mathcal{D}_{train}$  (*training set* 80%) e in un insieme di test  $\mathcal{D}_{test}$  (*test set* 20%), applicando una ripartizione stratificata (80/20) rispetto alla variabile target, con l'obiettivo di preservare la medesima proporzione tra pazienti sani e malati presente nel campione originale.

In particolare, le 303 osservazioni vengono suddivise in 242 campioni per il *training set* e 61 campioni per il *test set*:

$$|\mathcal{D}_{train}| = 242, |\mathcal{D}_{test}| = 61.$$

Successivamente, per la validazione interna del modello e la valutazione delle prestazioni su  $\mathcal{D}_{train}$ , è stata implementata la tecnica della *Stratified K-Fold Cross-Validation*. Analogamente al partizionamento iniziale, questa variante della validazione incrociata assicura che all'interno di ogni singola partizione (*fold*) venga rigorosamente mantenuta la proporzione originaria delle classi.

Come illustrato nella Figura 1, l'analisi della distribuzione delle classi a seguito della binarizzazione del target, ha evidenziato che il dataset non risulta particolarmente sbilanciato; questo impedisce al modello di sviluppare un bias verso una classe maggioritaria.

Nello specifico, il campione totale di 303 osservazioni è così distribuito:

- classe 0: 164 pazienti (54.1%)
- classe 1: 139 pazienti (45.9%)

L'analisi preliminare del dataset ha evidenziato la presenza di 6 valori mancanti (NaN), 4 valori mancanti nella variabile numerica *ca* e 2 valori mancanti nella

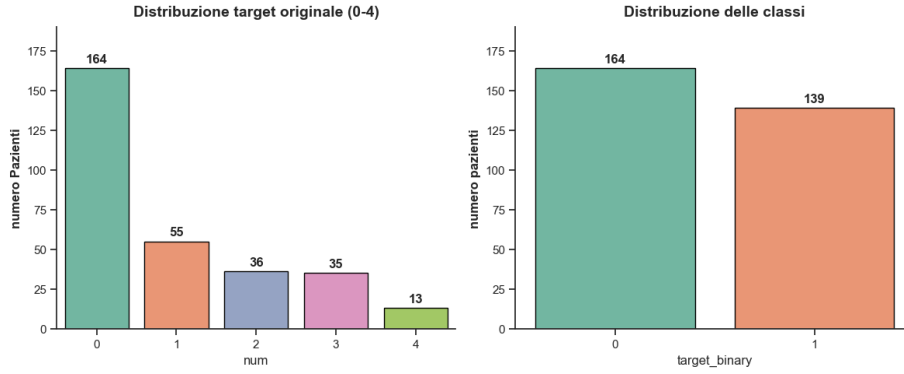


Figura 1: Distribuzione delle classi

variabile categorica *thal*. Nonostante si tratti di un numero ridotto, si è scelto di evitare l'eliminazione diretta delle relative righe, ma piuttosto di mantenere un approccio conservativo, in quanto il dataset contiene un totale di 303 osservazioni e l'esclusione avrebbe comportato una perdita di informazione per l'addestramento del modello.

A tal fine, si sono applicate delle tecniche di imputazione, in base alla natura intrinseca di ciascuna variabile coinvolta. Per la variabile quantitativa discreta *ca*, i valori mancanti sono stati imputati utilizzando la mediana calcolata sul campione di addestramento. Tale metrica risulta più robusta rispetto alla media, essendo meno sensibile all'eventuale presenza di outlier. Per quanto riguarda la variabile categorica *thal*, si è proceduto sostituendo i valori mancanti con la moda, assegnando così la categoria più frequente.

Per addestrare correttamente il modello di ottimizzazione, questi dati eterogenei devono essere convertiti in un formato puramente numerico. Sono state quindi applicate due trasformazioni:

- Le variabili categoriche sono state convertite in binarie tramite *One-Hot Encoding*.
- Le variabili continue sono state standardizzate (*Z-score*) per uniformare le scale di grandezza.

Lo spazio di output  $\mathcal{Y}$  (spazio delle *label*) è definito come:

$$\mathcal{Y} = \{0, 1\}$$

dove  $y_i = 0$  indica l'assenza di malattia cardiaca, mentre  $y_i = 1$  indica la presenza di patologia cardiaca, accorpando tutti gli stadi di gravità successivi.

Prima del *preprocessing*, le informazioni relative a ciascun paziente sono rappresentate da un vettore eterogeneo  $x_i$  costituito da  $d = 13$  variabili predittive

di tipo continuo, categorico e binario.

A seguito delle operazioni di imputazione dei dati mancanti, standardizzazione delle variabili continue e one-hot encoding delle variabili categoriche, l'intero dataset è rappresentato da una nuova matrice continua di dimensione  $D = 22$  (estesa rispetto a  $d = 13$  a causa dell'encoding delle variabili categoriche):

$$X \in \mathbb{R}^{303 \times 22}$$

Infine, per poter includere l'intercetta (*bias*) direttamente nei calcoli vettoriali, la matrice è stata espansa aggiungendo una prima colonna composta interamente da valori unitari. La matrice aumentata definitiva utilizzata per l'addestramento è:

$$X_{aug} \in \mathbb{R}^{303 \times 23}$$

## 2 Modello di classificazione: Regressione Logistica

Per affrontare il task di classificazione binaria descritto, il presente studio impiega il modello di *regressione logistica*.

Si tratta di un modello di classificazione utilizzato quando la variabile target è di natura categorica, assicurando che l'uscita ricada sempre nell'intervallo  $(0,1)$  attraverso l'uso della funzione Sigmoide  $\sigma : \mathbb{R} \rightarrow (0,1)$ , definita come segue:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

dove  $z = w^T x$ .

Si tratta di una funzione di attivazione non lineare, monotona crescente e differenziabile. Essa garantisce che l'output del modello possa essere rigorosamente interpretato come una probabilità.

### Spazio delle Ipotesi e formulazione del modello

Dal punto di vista teorico, l'addestramento di un modello di Machine Learning consiste nella ricerca della funzione ottima all'interno di un predefinito *Spazio delle Ipotesi* (*Hypothesis Space*)  $\mathcal{H}$ . Per la Regressione Logistica, tale spazio è costituito dall'insieme di tutte le funzioni lineari mappate attraverso la funzione sigmoide, parametrizzate dal vettore dei pesi:

$$\mathcal{H} = \{h_w(x) = \sigma(w^T x) \mid w \in \mathbb{R}^{D+1}\}$$

Per gestire implicitamente il termine di intercetta (*bias*), si adotta la convenzione di aumentare la matrice delle feature inserendo una colonna unitaria. Pertanto, per semplicità di notazione, assumeremo che ogni vettore delle feature  $x_i$  includa il termine unitario, cioè  $x_i = [1, x_{i1}, x_{i2}, \dots, x_{iD}]^T$  e il vettore dei parametri includa il bias ( $w \in \mathbb{R}^{D+1}$ ).

La probabilità a posteriori che il paziente sia affetto da patologia cardiaca dato il suo quadro clinico, è calcolata come:

$$\hat{y}_i = P(y_i = 1 \mid x_i; w) = \sigma(w^T x_i)$$

La probabilità complementare (assenza di malattia cardiaca) risulta essere:

$$P(y_i = 0 \mid x_i; w) = 1 - \hat{y}_i$$

Il modello di regressione logistica associa a ciascuna osservazione  $i$ -esima la probabilità stimata di appartenenza alla classe positiva:

$$\hat{y}_i = \sigma(w^T x_i)$$

In forma matriciale, l'equazione assume la seguente forma:

$$\hat{y} = \sigma(Xw),$$

dove  $\hat{y}$  è il vettore delle probabilità stimate,  $X$  è la matrice aumentata delle feature, e la funzione sigmoide è applicata componente per componente.

## 2.1 Regola di Decisione

Al fine di convertire l'output continuo  $\hat{y}_i$  restituito dal modello in una previsione di classe discreta, viene introdotta una soglia decisionale (*threshold*), impostata al valore di 0.5. La regola di classificazione per assegnare l'etichetta finale  $y_{pred,i}$  al paziente è definita come segue:

$$y_{pred,i} = \begin{cases} 1 & \text{se } \hat{y}_i \geq 0.5 \\ 0 & \text{se } \hat{y}_i < 0.5 \end{cases}$$

Dal punto di vista geometrico, adottare una soglia pari a 0.5 equivale a classificare l'istanza come positiva quando l'argomento della sigmoide è maggiore o uguale a 0, ovvero quando  $w^T x_i \geq 0$ . L'equazione  $w^T x_i = 0$  descrive l'iperpiano decisionale lineare che separa le due classi nello spazio delle feature  $\mathbb{R}^{D+1}$ .

## 2.2 Problema di finite-sum: minimizzazione del rischio empirico

L'obiettivo è determinare la presenza  $y = 1$  o l'assenza  $y = 0$  di una patologia coronarica a partire da un profilo clinico di un paziente rappresentato da un vettore di caratteristiche  $x \in \mathbb{R}^{D+1}$ .

Nel contesto dell'apprendimento supervisionato, l'addestramento della regressione logistica per il task di classificazione binaria si formula come un problema di ottimizzazione non vincolata di tipo finite-sum.

L'algoritmo di ottimizzazione deve determinare il vettore dei parametri ottimo  $w^* \in \mathbb{R}^{D+1}$  che minimizza il rischio empirico  $f(w)$  calcolato come la media delle loss sui singoli campioni del dataset, a cui viene aggiunto un termine di regolarizzazione  $L_2$  (Ridge) per prevenire l'overfitting:

$$w^* = \arg \min_{w \in \mathbb{R}^{D+1}} f(w)$$

con

$$f(w) = \frac{1}{N} \sum_{i=1}^N \ell_i(w) + \frac{\lambda}{2N} \|w_{reg}\|^2$$

dove  $\lambda > 0$  è l'iperparametro che controlla l'intensità della regolarizzazione e  $w_{reg} = [0, w_1, w_2, \dots, w_D]^T$  è il vettore dei pesi in cui il termine di bias è stato posto a zero (necessario per non regolarizzare l'intercetta).

In questo contesto, la loss  $\ell_i(w)$  rappresenta la Binary Cross-Entropy (o Log-Loss) valutata sulla singola osservazione  $(x_i, y_i)$ :

$$\ell_i(w) = -[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

dove  $\hat{y}_i$  è la probabilità stimata dal modello.

Sostituendo, il funzionale obiettivo completo risulta:

$$f(w) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] + \frac{\lambda}{2N} \|w_{reg}\|^2$$

Grazie alla linearità dell'operatore derivata, il gradiente di  $f(w)$  può essere espresso come la somma finita dei gradienti locali, evidenziando in modo esplicito la dipendenza dai pesi:

$$\nabla f(w) = \frac{1}{N} \sum_{i=1}^N \nabla \ell_i(w) + \frac{\lambda}{N} w_{reg} = \frac{1}{N} \sum_{i=1}^N x_i (\hat{y}_i - y_i) + \frac{\lambda}{N} w_{reg}$$

Questa impostazione teorica è stata fondamentale per guidare le scelte implementative nel codice, permettendo la vettorizzazione delle operazioni e garantendo un'elevata efficienza computazionale durante l'addestramento.

Sfruttando la struttura matriciale, il calcolo simultaneo del gradiente è stato implementato efficientemente traducendo l'equazione precedente nella sua forma compatta:

$$\nabla f(w) = \frac{1}{N} X^T (\hat{y} - y) + \frac{\lambda}{N} w_{reg}$$

Pertanto, a ogni iterazione dell'algoritmo di ottimizzazione, vengono calcolati la loss  $f(w)$ , il gradiente  $\nabla f(w)$  e il vettore delle probabilità predette  $\hat{y}$ .

Per l'implementazione degli algoritmi esatti del secondo ordine, si ricava la matrice hessiana  $H_f(w) \in \mathbb{R}^{(D+1) \times (D+1)}$ :

$$H_f(w) = \frac{1}{N} X^T S X + \frac{\lambda}{N} R$$

dove  $S \in \mathbb{R}^{N \times N}$  è la matrice diagonale con elementi  $S_{ii} = \hat{y}_i(1 - \hat{y}_i)$  e  $R \in \mathbb{R}^{(D+1) \times (D+1)}$  è la matrice identità in cui l'elemento in posizione  $(0,0)$  è stato posto a 0.

### 3 Addestramento del modello e Metodi di ottimizzazione

Durante la fase di addestramento, il modello impara a riconoscere le caratteristiche per distinguere i pazienti sani da quelli malati utilizzando il training set. Al termine dell'addestramento, il modello verrà messo alla prova sul test set e verrà valutata la sua capacità di classificazione.

In particolare, l'obiettivo di questa fase è determinare il vettore dei parametri ottimali della regressione logistica che minimizza la *loss function*  $f(w)$ , per poter massimizzare la capacità del modello di generalizzare quanto imparato dal training set su nuovi dati proposti.

Poiché il funzionale obiettivo dipende dal vettore dei parametri  $w$  e non presenta vincoli espliciti, il problema può essere formulato come un problema di ottimizzazione non vincolata:

$$\arg \min_{w \in \mathbb{R}^{D+1}} f(w)$$

L'addestramento viene eseguito attraverso il metodo *fit*, al quale vengono forniti in input la matrice delle feature del training set e il vettore delle label.

#### 3.1 Inizializzazione dei parametri

All'interno del metodo *fit*, a partire dalla matrice delle feature  $X$  viene costruita la matrice aumentata che contiene l'aggiunta di una colonna costituita da valori pari a 1, la quale consente di includere il termine di bias direttamente nel vettore dei parametri  $w$ .

All'inizio dell'ottimizzazione tutti i parametri del modello sono inizializzati a zero.

Nel presente lavoro è stata realizzata una classe custom, denominata *RegressioneLogistica*, nella quale l'algoritmo di ottimizzazione può essere selezionato tramite il parametro *metodo*. L'implementazione prevede la possibilità di scegliere uno dei diversi metodi iterativi: Steepest Descent, Newton, Newton Inesatto (N-CG), BFGS, L-BFGS.

La struttura generale di tali algoritmi iterativi prevede che ad ogni iterazione  $k$ , il vettore dei parametri venga aggiornato secondo la regola:

$$w_{k+1} = w_k + \alpha_k d_k$$

dove  $d_k$  rappresenta la direzione di discesa per la loss  $f$  e  $\alpha_k$  è il learning rate o stepsize lungo tale direzione.

Il presente studio analizza e implementa diverse strategie per la scelta della direzione  $d_k$  e per la scelta dello stepsize  $\alpha_k$ .

## 3.2 Scelta dello stepsize

Il problema della determinazione di  $\alpha_k$  si riduce allo studio della funzione univariata  $\phi(\alpha) = f(w_k + \alpha d_k)$ . Poiché la determinazione analitica del punto di minimo di  $\phi$  esatto lungo la direzione  $d_k$  (nota come *ricerca in linea esatta*) risulta computazionalmente onerosa, specialmente in problemi complessi, l'implementazione predilige una ricerca inesatta, la quale garantisce opportune proprietà di convergenza con minore sforzo computazionale.

### 3.2.1 Scelta della lunghezza del passo iniziale $\alpha_0$

La scelta dello stepsize iniziale  $\alpha_0$  nella ricerca in linea inesatta è un aspetto cruciale per l'efficienza computazionale del modello, poiché una buona stima può ridurre drasticamente il numero di valutazioni della loss e del gradiente, garantendo le proprietà di convergenza rapida degli algoritmi.

Nel codice implementato, la scelta di  $\alpha_0$  è stata ponderata in base alle caratteristiche di ciascun metodo. Per i metodi Newton e quasi-Newton, il passo  $\alpha_0 = 1$  viene sempre essere usato come lunghezza iniziale del passo di prova. Questa scelta garantisce che le lunghezze del passo unitario siano prese ogni volta che soddisfano le condizioni di terminazione e consente alle proprietà del rapido tasso di convergenza di questi metodi di avere effetto.

Per i metodi che non producono direzioni di ricerca ben scalate, come lo steepest descent e il metodo del gradiente coniugato, è importante utilizzare le informazioni attuali sul problema e l'algoritmo per formulare la stima iniziale. Pertanto, seguendo una strategia di interpolazione quadratica descritta da Nocedal e Wright, il passo iniziale è stato calcolato tramite interpolazione quadratica basata sui valori della loss precedente  $f_{k-1}$ , della loss attuale  $f_k$ , e la derivata direzionale attuale. Questa strategia produce  $\alpha_0 = \frac{2(f_k - f_{k-1})}{\nabla f_k^T d_k}$ . A livello implementativo, il primo passo viene scelto  $\alpha_0 = \frac{1}{\|\nabla f_0\|}$ , e nelle iterazioni successive viene utilizzata  $\alpha_0 = \frac{2(f_k - f_{k-1})}{\nabla f_k^T d_k}$  alla quale viene applicata un'euristica di correzione  $\alpha_0 = \min(1, 1.01\alpha_0)$  per impedire all'algoritmo di tentare passi iniziali eccessivamente ampi che potrebbero causare instabilità.

### 3.2.2 Ricerca inesatta: condizione di Armijo

L'obiettivo della ricerca inesatta è individuare un learning rate che fornisca un sufficiente decremento della loss function lungo la direzione di ricerca.

A tale scopo, l'algoritmo di **Backtracking-Armijo** impone che il decremento di  $f$  sia proporzionale ad  $\alpha$  e alla derivata direzionale iniziale. Fissato un iperparametro di controllo  $c_1 \in (0, 1)$ , la condizione di sufficiente decremento della regola di Armijo richiede che:

$$f(w_k + \alpha d_k) \leq f(w_k) + c_1 \alpha \nabla f(w_k)^T d_k$$

Geometricamente, tale condizione assicura che il valore della funzione nel nuovo punto si collochi al di sotto di una retta secante passante per  $f(w_k)$ .

L'algoritmo di backtracking procede inizializzando lo stepsize iniziale al valore  $\alpha_0$  (scelto in base all'algoritmo utilizzato), e lo riduce iterativamente di un coefficiente di riduzione  $\tau$ , impostato a 0.5, finché la disuguaglianza non è soddisfatta.

Essendo la loss function sufficientemente regolare e il  $\nabla f$  è globalmente lipschitziano, l'algoritmo di Backtracking-Armijo line-search arriva a terminazione in un numero finito di passi.

Un algoritmo di discesa che utilizza tale strategia di Backtracking-Armijo converge globalmente, raggiungendo un punto stazionario in un numero finito di passi, o convergendo asintoticamente ad esso.

L'esecuzione del metodo di ottimizzazione si interrompe se la norma euclidea del gradiente scende al di sotto di una soglia di tolleranza prefissata  $\|\nabla f(w_k)\| < tol$ , confermando il raggiungimento di un punto stazionario.

### 3.2.3 Strategie migliorative: Condizioni di Wolfe e Armijo-Goldstein

Il limite principale della sola condizione di Armijo è la mancata garanzia che il passo non diventi troppo piccolo, il che potrebbe compromettere i progressi dell'algoritmo. Strategie migliorative per limitare questo fenomeno, che prevedono il decremento della loss ed euristiche per evitare che lo stepsize diventi troppo piccolo, introducono criteri aggiuntivi: le condizioni di Wolfe e di Armijo-Goldstein, basati su due costanti  $0 < c_1, c_2 < 1$ .

#### Le condizioni di Wolfe

1. Sufficiente decremento:

$$f(w_k + \alpha_k d_k) \leq f(w_k) + c_1 \alpha_k \nabla f(w_k)^T d_k$$

2. Condizione di curvatura:

$$\nabla f(w_k + \alpha_k d_k)^T d_k \geq c_2 \nabla f(w_k)^T d_k$$

#### Le Condizioni di Wolfe forti (*Strong Wolfe Conditions*)

Queste condizioni combinano il sufficiente decremento di Armijo con un vincolo esplicito sulla derivata direzionale nel nuovo punto.

1. Sufficiente decremento:

$$f(w_k + \alpha_k d_k) \leq f(w_k) + c_1 \alpha_k \nabla f(w_k)^T d_k$$

2. Condizione di curvatura:

$$|\nabla f(w_k + \alpha_k d_k)^T d_k| \leq c_2 |\nabla f(w_k)^T d_k|$$

### Le Condizioni di Armijo-Goldstein

Queste vincolano il valore della funzione dall'alto e dal basso mediante due rette secanti distinte:

1. Limite superiore:

$$f(w_k + \alpha_k d_k) \leq f(w_k) + c_1 \alpha_k \nabla f(w_k)^T d_k$$

2. Limite inferiore:

$$f(w_k + \alpha_k d_k) \geq f(w_k) + c_2 \alpha_k \nabla f(w_k)^T d_k$$

Sebbene più deboli rispetto a quelle di Wolfe (poiché non garantiscono l'esclusione di minimi locali per il passo  $\alpha$ ), esse risultano computazionalmente vantaggiose costringendo l'algoritmo a mantenere uno *step-size* sufficientemente esteso, impedendo riduzioni premature del passo durante la ricerca unidimensionale.

## 3.3 Metodo del gradiente

Il *metodo della discesa del gradiente* (o *steepest descent*) è un algoritmo del primo ordine ed appartiene alla classe dei metodi line-search. Ad ogni passo, esso si muove lungo la direzione di ricerca dell'antigradiente:  $d_k = -\nabla f(w_k)$ . La regola di aggiornamento dello steepest descent è:

$$w_{k+1} = w_k - \alpha_k \nabla f(w_k)$$

dove  $\alpha_k$  è lo stepsize (o learning rate), determinato mediante una strategia di line search inesatta (condizione di Armijo), al fine di garantire una sufficiente decrescita della loss.

Il vantaggio principale dello steepest descent risiede nel suo basso costo computazionale, poiché richiede solo il calcolo del gradiente  $\nabla f(w_k)$ , con un costo di  $O(N \cdot d)$ , dove  $N$  è il numero di campioni e  $d$  è il numero delle feature, per ogni iterazione.

Il limite risiede nel suo tasso di convergenza esclusivamente lineare. In prossimità del minimo, specialmente in presenza di problemi con elevato numero di condizionamento (geometrie a "valle stretta"), la traiettoria di discesa tende a oscillare generando un andamento a "zig-zag". Tale inefficienza costringe l'algoritmo a compiere un elevato numero di iterazioni prima di poter soddisfare un criterio di arresto sufficiente sulla norma del gradiente.

### 3.3.1 Convergenza Globale e Teorema di Zoutendijk

Per analizzare la convergenza globale di una generica classe di metodi line-search, si fa riferimento al classico risultato teorico dovuto a Zoutendijk, formulato sotto l'ipotesi che lo stepsize soddisfi le condizioni di Wolfe.

**Teorema 3.1** (Teorema di Zoutendijk).

Sia  $f \in C^1(\mathbb{R}^d)$  e limitata inferiormente. Si assuma che il gradiente  $\nabla f$  sia Lipschitziano. Sia  $\{w_k\}_{k \in \mathbb{N}}$  la successione generata dall'algoritmo di discesa:

$$w_{k+1} = w_k + \alpha_k d_k,$$

dove  $d_k \in \mathbb{R}^d$  è una direzione di discesa tale che  $\nabla f(w_k)^T d_k < 0$ , e lo step size  $\alpha_k$  soddisfa le condizioni di Wolfe. Allora vale la seguente condizione:

$$\sum_{k \geq 0} \cos(\theta_k)^2 \|\nabla f(w_k)\|^2 < +\infty \quad (1)$$

con

$$\cos \theta_k = \frac{-\nabla f_k^T d_k}{\|\nabla f_k\| \|d_k\|}$$

e  $\theta_k$  è l'angolo formato tra  $d_k$  e la direzione di massima discesa  $-\nabla f(w_k)$ .

*Dimostrazione.* Dalla seconda condizione di Wolfe e dall'aggiornamento  $w_{k+1} = w_k + \alpha_k d_k$ , si ha:

$$(\nabla f_{k+1} - \nabla f_k)^T d_k \geq (c_2 - 1) \nabla f_k^T d_k$$

mentre la condizione di lipschitzianità del gradiente implica che:

$$(\nabla f_{k+1} - \nabla f_k)^T d_k \leq \alpha_k L \|d_k\|^2$$

Combinando queste due disuguaglianze, si ottiene:

$$\alpha_k \geq \frac{c_2 - 1}{L} \frac{\nabla f_k^T d_k}{\|d_k\|^2}$$

Sostituendo questa stima nella prima condizione di Wolfe ( $f_{k+1} \leq f_k + c_1 \alpha_k \nabla f_k^T d_k$ ), otteniamo:

$$f_{k+1} \leq f_k - c_1 \frac{1 - c_2}{L} \frac{(\nabla f_k^T d_k)^2}{\|d_k\|^2}$$

Sfruttando la definizione di  $\cos \theta_k$ , possiamo riscrivere la precedente relazione come:

$$f_{k+1} \leq f_k - c \cos^2 \theta_k \|\nabla f_k\|^2$$

dove  $c = \frac{c_1(1-c_2)}{L} > 0$ . Sommando questa espressione su tutti gli indici da 0 a  $k$ , si ricava:

$$f_{k+1} \leq f_0 - c \sum_{j=0}^k \cos^2 \theta_j \|\nabla f_j\|^2$$

Poiché  $f$  è limitata inferiormente, la quantità  $f_0 - f_{k+1}$  è limitata superiormente da una costante positiva per ogni  $k$ . Di conseguenza, passando al limite per  $k \rightarrow \infty$ , si ottiene:

$$\sum_{k=0}^{\infty} \cos(\theta_k)^2 \|\nabla f(w_k)\|^2 < \infty$$

□

Risultati analoghi valgono anche quando si utilizzano le condizioni di Goldstein o le condizioni di Wolfe forti al posto delle condizioni di Wolfe ordinarie. Per tutte queste strategie, la selezione della lunghezza del passo implica la disuguaglianza precedente (1), che prende il nome di *condizione di Zoutendijk*.

La condizione di Zoutendijk implica che:

$$\lim_{k \rightarrow \infty} \cos(\theta_k)^2 \|\nabla f(w_k)\|^2 = 0 \quad (2)$$

e questo limite consente di derivare i risultati di convergenza globale per gli algoritmi line search.

Se la scelta della direzione  $d_k$  assicura che l'angolo  $\theta_k$  sia uniformemente separato da  $90^\circ$ , esiste una costante positiva  $\delta$  tale che:

$$\cos \theta_k \geq \delta > 0, \quad \forall k \quad (3)$$

allora segue che:

$$\lim_{k \rightarrow \infty} \|\nabla f(w_k)\| = 0 \quad (4)$$

ovvero un algoritmo di discesa con line-search che soddisfa le condizioni di Wolfe converge a un punto stazionario di  $f$ .

In altri termini, la norma del gradiente converge a zero purché le direzioni di ricerca non diventino mai quasi ortogonali al gradiente.

In particolare, nel caso specifico dello steepest descent, la direzione di ricerca è scelta sempre pari all'antigradiente:

$$d_k = -\nabla f(w_k)$$

di conseguenza, essa coincide esattamente con la direzione di massima pendenza. L'angolo  $\theta_k$  tra la direzione scelta  $d_k$  e l'antigradiente  $-\nabla f(w_k)$  risulta pertanto sempre nullo:

$$\theta_k = 0 \implies \cos(\theta_k) = 1$$

In questo scenario, la condizione  $\cos(\theta_k) \geq \delta > 0$  è automaticamente e banalmente soddisfatta a ogni iterazione, potendo scegliere  $\delta = 1$ .

Pertanto, se il passo  $\alpha_k$  viene determinato mediante una *line search* sufficientemente regolare, e le restanti ipotesi del teorema risultano verificate, la condizione di Zoutendijk assicura che:

$$\lim_{k \rightarrow \infty} \|\nabla f(w_k)\| = 0.$$

### 3.4 Metodo di Newton

Il metodo di Newton è un algoritmo del secondo ordine che approssima localmente la loss  $f(w)$  tramite il suo sviluppo in serie di Taylor troncato al secondo ordine nell'intorno dell'iterata corrente  $w_k \approx w^*$  (ipotesi di localizzazione):

$$Q(w) = f(w_k) + \nabla f(w_k)^T(w - w_k) + \frac{1}{2}(w - w_k)^T H_f(w_k)(w - w_k)$$

Come nuova approssimazione  $w_{k+1}$  della soluzione  $w^*$ , si sceglie il punto che minimizza il modello quadratico  $Q(w)$ . Tale punto deve necessariamente annullare il gradiente di  $Q$  (condizione necessaria di stazionarietà):

$$\nabla Q(w) = \nabla f(w_k) + H_f(w_k)(w - w_k) = 0$$

Definendo il vettore  $d = w - w_k$ , la relazione precedente si riscrive come il sistema lineare:

$$H_f(w_k)d = -\nabla f(w_k)$$

Se la matrice hessiana definita positiva, essa è invertibile, pertanto dal sistema si ricava la direzione di discesa di Newton:

$$d_k^N = -[H_f(w_k)]^{-1} \nabla f(w_k)$$

Il Metodo di Newton sfrutta l'informazione del secondo ordine (la curvatura) per calcolare l'aggiornamento dei pesi. L'iterazione del metodo di Newton è definita come:

$$w_{k+1} = w_k + \alpha_k d_k^N$$

dove  $\alpha_k$  è lo step size calcolato tramite line search.

Dal punto di vista computazionale, il calcolo della direzione avviene risolvendo il sistema lineare associato, utilizzando la fattorizzazione  $LU$  mediante *numpy.linalg.solve*, che richiede un costo di  $O(d^3)$  per iterazione, mentre la memorizzazione della matrice hessiana richiede  $O(d^2)$  memoria.

Diversamente dal metodo del gradiente, per il metodo di Newton con line search, il valore iniziale di  $\alpha$  viene impostato a 1 e viene ridotto solo se tale valore non produce una sufficiente decrescita della funzione obiettivo. Una volta sufficientemente vicini alla soluzione, l'accettazione del passo unitario ( $\alpha_k = 1$ ) consente di recuperare la convergenza locale di ordine quadratico del metodo di Newton puro.

### 3.5 Metodo di Newton inesatto: Newton-CG

Il metodo di Newton classico richiede la risoluzione del sistema lineare:

$$H_f(w_k)d_k^N = -\nabla f(w_k)$$

ad ogni iterazione, operazione che comporta un costo computazionale pari a  $O(d^3)$ .

Per ridurre tale onere computazionale si utilizza il metodo di *Newton-CG*, noto anche come *metodo di Newton troncato*, che risolve il sistema lineare tramite il *metodo del gradiente coniugato*. Invece della soluzione esatta del sistema, si calcola una direzione di ricerca approssimata  $d_k$  che soddisfa il sistema fino a una determinata tolleranza sul residuo del sistema.

Il criterio di arresto per le iterazioni interne del CG, si basa sulla norma del residuo del sistema  $r_j = B_k z_j + \nabla f(w_k)$ , dove  $B_k = H_f(w_k)$  e  $z_j$  è la direzione accumulata al passo  $j$ . Il ciclo si arresta quando la norma del residuo diventa inferiore a una tolleranza  $\epsilon_k$ :

$$\|r_{j+1}\| < \epsilon_k, \quad \text{con } \epsilon_k = \eta_k \|\nabla f(w_k)\|$$

dove il parametro  $\eta_k \in (0, 1)$  rappresenta il *termine di forzamento* (*forcing term*), il quale controlla l'accuratezza della soluzione, determinando la velocità di convergenza dell'algoritmo dei metodi di Newton inesatti.

La convergenza locale si ottiene assicurandosi che  $\eta_k$  sia limitato superiormente e si mantenga strettamente minore di 1.

Se si forza  $\eta_k$  a tendere a zero man mano che si avvicina all'ottimo, il metodo accelera ottenendo una convergenza superlineare.

Invece, se l'hessiana è lipschitziana, e se la tolleranza  $\eta_k = O(\|\nabla f(w_k)\|)$ , il metodo è quadratico. Tali risultati di convergenza hanno natura strettamente locale. Essi assumono, infatti, che le iterate si trovino già in un intorno sufficientemente piccolo del minimo  $w^*$ .

Per ottenere convergenza superlineare,  $\eta_k$  viene aggiornato dinamicamente attraverso la seguente scelta:  $\eta_k = \min \left( 0.5, \sqrt{\|\nabla f(w_k)\|} \right)$ .

Inoltre, poiché il metodo del CG è progettato per risolvere sistemi definiti positivi, l'iterazione interna viene interrotta non appena viene rilevata una direzione di curvatura non positiva, questo serve al fine di garantire che la direzione  $d_k$  sia di discesa. A tale scopo, a ogni passo del CG, viene monitorata la curvatura lungo la direzione di ricerca  $d_j$ :

$$d_j^T B_k d_j \leq 0$$

Se tale condizione si verifica alla prima iterazione ( $j = 0$ ), l'algoritmo interrompe il CG e restituisce la direzione dell'antigradiente ( $d_k = -\nabla f(w_k)$ ); se invece si verifica in una delle iterazioni successive ( $j > 0$ ), il ciclo viene arrestato restituendo l'ultima direzione accumulata fino a quel momento ( $d_k = z_j$ ).

In presenza di curvatura strettamente positiva, l'algoritmo aggiorna la soluzione e il residuo:

$$\alpha_j = \frac{r_j^T r_j}{d_j^T B_k d_j}, \quad z_{j+1} = z_j + \alpha_j d_j, \quad r_{j+1} = r_j + \alpha_j B_k d_j$$

Se la norma del nuovo residuo verifica il criterio di arresto:  $\|r_{j+1}\| < \epsilon_k$ , il ciclo termina con successo e viene restituita la direzione  $d_k = z_{j+1}$ . Altrimenti, viene aggiornata la direzione di ricerca coniugata  $d_{j+1} = -r_{j+1} + \beta_{j+1}d_j$  con  $\beta_{j+1} = \frac{r_{j+1}^T r_{j+1}}{r_j^T r_j}$  e si prosegue fino a un massimo di iterazioni interne. Infine, ottenuta la direzione  $d_k$  dalla procedura CG, l'aggiornamento dei pesi avviene secondo la regola:

$$w_{k+1} = w_k + \alpha_k d_k$$

dove lo step size  $\alpha_k$  viene determinato tramite *line search* con procedura di *backtracking* basata sulle condizioni di Armijo.

### 3.6 Metodi quasi-Newton: BFGS, L-BFGS

I metodi di quasi-Newton sono stati introdotti per ovviare all'eccessivo carico computazionale dei metodi di Newton puri, senza rinunciare alla convergenza dell'algoritmo. Essi offrono una valida alternativa in quanto non richiedono il calcolo della matrice hessiana, ma riescono ad avere una velocità di convergenza superlineare.

I metodi quasi-Newton, richiedono solo la valutazione del gradiente della funzione obiettivo da fornire ad ogni iterazione.

L'algoritmo quasi-Newton più diffuso è il metodo BFGS, che prende il nome dai suoi ideatori Broyden, Fletcher, Goldfarb e Shanno.

L'implementazione si basa sull'aggiornamento iterativo della matrice  $B_k$ , che approssima l'inversa dell'hessiana. Gli aggiornamenti della matrice  $B_k$  sfruttano il cambiamento del gradiente  $\nabla f(w_k)$  tra due iterate successive che fornisce informazioni sulla derivata seconda di  $f$  lungo la direzione di ricerca.

In un intorno di  $w_k$ , la loss viene approssimata da un modello quadratico locale in termini di uno spostamento  $p$ :

$$m_k(p) = f(w_k) + \nabla f(w_k)^T p + \frac{1}{2} p^T B_k p$$

dove  $B_k$  è una matrice simmetrica e definita positiva, che verrà aggiornata ad ogni iterazione.

Si noti che il valore della funzione e il gradiente di questo modello nell'origine  $p = 0$  corrispondono rispettivamente a  $f(w_k)$  e  $\nabla f(w_k)$ .

Minimizzando il modello quadratico imponendo la condizione di stazionarietà  $\nabla m_k(p) = 0$ , si ottiene:

$$\nabla f(w_k) + B_k p = 0 \iff p_k = -B_k^{-1} \nabla f(w_k)$$

e la nuova iterazione è:

$$w_{k+1} = w_k + \alpha_k p_k$$

dove lo stepsize  $\alpha_k$  viene individuato mediante backtracking richiedendo il soddisfacimento delle condizioni di Wolfe. Questa iterazione è molto simile al metodo di ricerca in linea di Newton; la differenza chiave è che viene utilizzata un'approssimazione dell'hessiana  $B_k$  al posto dell'hessiana esatta.

Invece di calcolare nuovamente  $B_k$  ad ogni iterazione, Davidon ha proposto di aggiornarla in modo semplice per tenere conto della curvatura misurata durante il passo più recente. Supponiamo di aver generato una nuova iterata  $w_{k+1}$  e desideriamo costruire un nuovo modello quadratico della forma:

$$m_{k+1}(p) = f(w_{k+1}) + \nabla f(w_{k+1})^T p + \frac{1}{2} p^T B_{k+1} p$$

Imponendo a  $B_{k+1}$  che il gradiente di  $m_{k+1}$  coincida con gradiente della funzione obiettivo  $f$  nelle ultime due iterate  $w_k$  e  $w_{k+1}$ .

Nel punto corrente  $w_{k+1}$ , che nel sistema di riferimento locale del modello corrisponde a uno spostamento nullo ( $p = 0$ ), tale condizione è soddisfatta automaticamente per costruzione, poiché  $\nabla m_{k+1}(0) = \nabla f(w_{k+1})$ .

Per imporre la coincidenza anche nel punto precedente  $w_k$  è necessario valutare il gradiente del modello in corrispondenza dello spostamento inverso  $\hat{p} = -\alpha_k p_k$  sulla curva di livello.

Sostituendo questo specifico spostamento nell'espressione del gradiente del modello, imponiamo che il risultato coincida con  $\nabla f(w_k)$

$$\nabla m_{k+1}(-\alpha_k p_k) = \nabla f(w_{k+1}) - \alpha_k B_{k+1} p_k = \nabla f(w_k)$$

Riordinando i termini, si ottiene:

$$B_{k+1} \alpha_k p_k = \nabla f(w_{k+1}) - \nabla f(w_k)$$

Per semplificare la notazione è utile definire i vettori di variazione degli spostamenti e del gradiente :

$$s_k = w_{k+1} - w_k = \alpha_k p_k, \quad y_k = \nabla f(w_{k+1}) - \nabla f(w_k)$$

sostituendoli, l'equazione diventa:

$$B_{k+1} s_k = y_k$$

che rappresenta l'**equazione secante**.

Moltiplicando a sinistra l'equazione secante per  $s_k^T$  si ottiene:

$$s_k^T B_{k+1} s_k = s_k^T y_k$$

Affinché la matrice  $B_{k+1}$  sia definita positiva (ovvero  $s_k^T B_{k+1} s_k > 0$  per ogni  $s_k$  diverso da 0, è necessario che valga la **condizione di curvatura**:

$$s_k^T y_k > 0$$

Pertanto la ricerca dello stepsize con le condizioni di Wolfe assicura automaticamente la validità della condizione di curvatura.

Tuttavia, l'equazione secante e i vincoli di simmetria e definita positività non sono sufficienti per determinare univocamente la matrice  $B_{k+1}$ , lasciando gradi di libertà nella sua costruzione. Per superare questo, si sceglie la matrice  $B_{k+1}$  che minimizza la distanza della matrice precedente  $B_k$  secondo una norma di Frobenius ponderata.

Da questo principio derivano i due principali metodi quasi-Newton:

- Metodo DFP: minimizza la distanza tra le matrici hessiane  $B$  e  $B_k$  soggetta a vincoli di simmetria e soddisfacimento dell'equazione secante.
- Metodo BFGS: applica la minimizzazione direttamente all'inversa dell'hessiana  $H_k = B_k^{-1}$ , risolvendo:

$$\begin{aligned} \min_H \quad & \|H - H_k\| \\ \text{s.t.} \quad & H = H^T, \quad Hy_k = s_k \end{aligned}$$

Definendo lo scalare  $\rho_k = \frac{1}{y_k^T s_k}$ , la soluzione unica fornisce l'aggiornamento del metodo BFGS:

$$H_{k+1} = (I - \rho_k s_k y_k^T) H_k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T$$

è un aggiornamento a rango due che garantisce la simmetria e la definita positività della matrice.

Lavorare direttamente con  $H_k$  consente di calcolare la direzione di ricerca  $d_k = -H_k \nabla f_k$  tramite una semplice moltiplicazione matrice-vettore, riducendo il costo per iterazione a  $O(d^2)$  senza dover risolvere sistemi lineari  $O(d^3)$  e inoltre garantisce una velocità di convergenza superlineare.

L'implementazione integra inoltre due specifiche euristiche. Alla prima iterazione, la matrice identità  $H_0$  viene scalata per il fattore  $\gamma_0 = \frac{s_0^T y_0}{y_0^T y_0}$ . In secondo luogo, la line search utilizza  $\alpha = 1$  come stepsize iniziale, poiché tale lunghezza alla fine verrà sempre accettata (sotto determinate condizioni), producendo in tal modo una convergenza superlineare del metodo BFGS.

Un limite di BFGS è rappresentato dal costo associato alla memorizzazione della matrice hessiana, che richiede  $O(d^2)$  memoria rende il metodo meno conveniente per problemi ad alta dimensionalità. Per tali problemi si preferisce ricorrere alla variante L-BFGS, che conserva solo un numero limitato  $m$  di vettori recenti e riduce l'occupazione di memoria a  $O(m \cdot d)$  con  $m \ll d$ .

### 3.6.1 Convergenza superlineare dei metodi Quasi-Newton

Prima di definire le condizioni di convergenza, è essenziale sottolineare un risultato chiave dei metodi Quasi-Newton: se la direzione di ricerca di un metodo

Quasi-Newton approssima sufficientemente bene la direzione di Newton, il passo unitario  $\alpha_k = 1$  soddisferà automaticamente le condizioni di Wolfe quando le iterate si avvicinano alla soluzione. Il seguente teorema formalizza questo concetto, stabilendo il criterio esatto che la direzione deve soddisfare per ottenere una convergenza superlineare.

**Teorema 3.2.** *Sia  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  una funzione di classe  $C^2$  e si consideri un metodo Quasi-Newton con direzione di ricerca:*

$$d_k = -B_k^{-1} \nabla f(w_k)$$

dove  $B_k$  è una matrice simmetrica e definita positiva, e assumendo una ricerca lungo la linea (line search) che soddisfa le condizioni di Wolfe, con  $c_1 \leq \frac{1}{2}$ . Si supponga che la successione  $\{w_k\}$  converga a un punto  $w^*$  tale che  $\nabla f(w^*) = 0$  e  $H_f(w^*) \succ 0$ . Se la direzione di ricerca soddisfa la condizione:

$$\lim_{k \rightarrow \infty} \frac{\|(B_k - H_f(w^*))d_k\|}{\|d_k\|} = 0 \quad (5)$$

detta **condizione di Dennis-Moré**, allora il passo unitario  $\alpha_k = 1$  risulta ammissibile per  $k$  sufficientemente grande di un certo indice  $k_0$ ; e se  $\alpha_k = 1$  per ogni  $k > k_0$ , la successione  $\{w_k\}$  converge a  $w^*$  in modo superlineare.

La condizione di Dennis-Moré esprime un concetto fondamentale per i metodi Quasi-Newton: una velocità di convergenza superlineare può essere ottenuta anche se la successione delle matrici  $B_k$  non converge alla hessiana  $H_f(w^*)$ , ma è sufficiente che le matrici  $B_k$  diventino approssimazioni sempre più accurate della hessiana lungo le direzioni di ricerca  $d_k$ . Tale condizione è necessaria e sufficiente per la convergenza superlineare dei metodi Quasi-Newton.

In particolare, se  $\alpha_k = 1$  per ogni  $k$ , vale il viceversa: la convergenza superlineare si verifica se e solo se è soddisfatta la condizione di Dennis-Moré.

**Teorema 3.3** (Convergenza superlineare del metodo BFGS). *Sia  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  una funzione di classe  $C^2$  e si supponga che la successione  $\{w_k\}$  generata dal metodo BFGS converga a un minimo  $w^*$ . Si assuma inoltre che la matrice Hessiana di  $f$  sia Lipschitziana in un intorno di  $w^*$ , ossia che esista una costante  $L > 0$  tale che:*

$$\|H_f(w) - H_f(w^*)\| \leq L\|w - w^*\|$$

per ogni  $w$  sufficientemente vicino a  $w^*$ . Se, inoltre:

$$\sum_{k=1}^{\infty} \|w_k - w^*\| < \infty$$

allora la successione delle iterate  $\{w_k\}$  converge a  $w^*$  con velocità superlineare.

### 3.6.2 Metodi Limited-memory quasi-Newton

I metodi quasi-Newton a memoria limitata sono efficaci per la risoluzione di problemi di ottimizzazione su larga scala, nei quali la matrice hessiana non può essere calcolata a un costo ragionevole o quando non si presenta in forma sparsa.

Essi mantengono approssimazioni semplici e compatte delle matrici hessiane: invece di memorizzare approssimazioni di matrici dense  $n \times n$ , memorizzano solo un numero ristretto di vettori di lunghezza  $n$  per rappresentare l'approssimazione desiderata. Pur operando con requisiti di memoria ridotti, riescono solitamente a garantire un tasso di convergenza lineare.

### 3.6.3 L-BFGS

Tra le diverse varianti proposte in letteratura, l'algoritmo *L-BFGS* (*Limited-memory BFGS*) è il più noto e utilizzato. L'idea principale di questo metodo consiste nello sfruttare le informazioni sulla curvatura relative alle iterazioni più recenti per costruire l'approssimazione dell'hessiana. Le informazioni sulla curvatura più remote, che hanno meno probabilità di essere rilevanti per il comportamento effettivo dell'hessiana nell'iterazione corrente, vengono scartate allo scopo di risparmiare spazio di memoria.

A partire dal metodo BFGS:

$$w_{k+1} = w_k - \alpha_k H_k \nabla f_k$$

dove  $H_k$  è aggiornata a ogni iterazione mediante la formula:

$$H_{k+1} = V_k^T H_k V_k + \rho_k s_k s_k^T$$

dove:

$$\rho_k = \frac{1}{y_k^T s_k}, \quad V_k = I - \rho_k y_k s_k^T$$

con  $s_k = w_{k+1} - w_k$ ,  $y_k = \nabla f_{k+1} - \nabla f_k$  che rappresentano rispettivamente lo spostamento tra due iterate consecutive e la corrispondente variazione del gradiente.

L-BFGS evita di costruire esplicitamente la matrice  $H_k$ , ma memorizza implicitamente una versione modificata di  $H_k$ , conservando un certo numero  $m$  di coppie di vettori  $\{s_i, y_i\}$ .

Una volta calcolata la nuova iterata, la nuova coppia  $\{s_k, y_k\}$  va a sostituire quella più vecchia all'interno della memoria. Se si raggiunge il limite massimo  $m$  di coppie memorizzabili, la coppia più vecchia viene eliminata. In questo modo, l'insieme delle coppie di vettori include informazioni sulla curvatura delle  $m$  iterazioni più recenti. In letteratura, si suggeriscono valori di  $m$  tra 3 e 20,

per ottenere risultati soddisfacenti. Per questo elaborato si è scelto di utilizzare  $m = 10$ , mantenendo quindi in memoria le ultime dieci coppie di vettori.

All'iterazione  $k$ , l'iterata corrente è  $w_k$  e l'insieme delle coppie di vettori è data da  $\{s_i, y_i\}$  per  $i = k - m, \dots, k - 1$ .

Si sceglie una certa approssimazione iniziale dell'Hessiana  $H_k^0$  (inversa all'iterazione  $k$ ) e troviamo, tramite l'applicazione ripetuta della formula di aggiornamento, che  $H_k$  assume la seguente forma:

$$\begin{aligned} H_k = & V_{k-1}^T \cdots V_{k-m}^T H_k^0 (V_{k-m} \cdots V_{k-1}) \\ & + \rho_{k-m} V_{k-1}^T \cdots V_{k-m+1}^T s_{k-m} s_{k-m}^T (V_{k-m+1} \cdots V_{k-1}) \\ & + \rho_{k-m+1} V_{k-1}^T \cdots V_{k-m+2}^T s_{k-m+1} s_{k-m+1}^T (V_{k-m+2} \cdots V_{k-1}) \\ & + \cdots \\ & + \rho_{k-1} s_{k-1} s_{k-1}^T. \end{aligned}$$

Da questa espressione possiamo derivare una procedura ricorsiva per calcolare il prodotto  $H_k \nabla f_k$  in modo efficiente, sfruttando la cosiddetta *ricorsione a due cicli* che sfrutta gli ultimi  $m$  vettori per calcolare il prodotto matrice-vettore in  $O(m \cdot d)$  operazioni.

L'algoritmo si articola in due fasi sequenziali:

- primo ciclo (all'indietro): partendo dal gradiente  $q = \nabla f_k$  e procedendo con un ciclo a ritroso dall'iterazione corrente fino a quella più remota  $k - m$ , l'algoritmo calcola dei coefficienti  $\alpha_i = \rho_i s_i^T q$  e  $q = q - \alpha_i y_i$ , in questo modo il gradiente corrente viene progressivamente corretto usando le informazioni di curvatura memorizzate  $(s_i, y_i)$  nelle ultime  $m$  iterazioni;
- terminato il primo ciclo, si applica l'approssimazione iniziale  $r = H_k^0 q$ , dove  $H_k^0$  è stata scelta come  $\gamma_k I$ .
- secondo ciclo (in avanti): percorrendo le coppie memorizzate, dalla più vecchia alla più recente, ossia per  $i = k - m, \dots, k - 1$ . Per ogni coppia si calcola  $\beta_i = \rho_i y_i^T r$ , e si aggiorna il vettore secondo  $r = r + s_i(\alpha_i - \beta_i)$ . Al termine del secondo ciclo si ottiene il prodotto  $r = H_k \nabla f_k$ , da cui viene ricavata la direzione di ricerca  $d_k = -r$ .

**Algoritmo** (Ricorsione a due loop L-BFGS)

```

 $q \leftarrow \nabla f_k;$ 
for  $i = k - 1, k - 2, \dots, k - m$ 
     $\alpha_i \leftarrow \rho_i s_i^T q;$ 
     $q \leftarrow q - \alpha_i y_i;$ 
end (for)
 $r \leftarrow H_k^0 q;$ 
for  $i = k - m, k - m + 1, \dots, k - 1$ 
     $\beta \leftarrow \rho_i y_i^T r;$ 
     $r \leftarrow r + s_i(\alpha_i - \beta);$ 
end (for)
stop con il risultato  $H_k \nabla f_k = r.$ 

```

La ricorsione a due cicli ha un costo computazionale dell'ordine di  $O(m \cdot d)$ , dove  $m$  è il numero di coppie di correzione conservate e  $d$  la dimensione del problema. Oltre ad essere economica, questa ricorsione ha il vantaggio che la moltiplicazione per la matrice iniziale  $H_k^0$  è isolata dal resto dei calcoli, permettendo di essere scelta liberamente e di farla variare tra un'iterazione e l'altra.

Un metodo per scegliere  $H_k^0$  che si è dimostrato efficace nella pratica è impostare  $H_k^0 = \gamma_k I$ , dove  $\gamma_k = \frac{s_{k-1}^T y_{k-1}}{y_{k-1}^T y_{k-1}}$ . Questa scelta aiuta ad assicurare che la direzione di ricerca  $d_k$  sia ben scalata e, di conseguenza, la lunghezza del passo  $\alpha_k = 1$  venga accettata nella maggior parte delle iterazioni. A tal fine, nell'implementazione proposta, la line search è stata basata sulle condizioni di Wolfe, requisito fondamentale affinché l'aggiornamento BFGS sia stabile.

L'algoritmo BFGS richiede ad ogni iterazione la chiede una direzione di ricerca, calcola l'ampiezza del passo ottimale  $\alpha_k$  tramite le condizioni di Wolfe e aggiorna di conseguenza i pesi del modello. La sua funzione distintiva risiede nella gestione dinamica della memoria: invece di accumulare l'intera storia dell'addestramento, mantiene una coda aggiornata delle ultime  $m$  variazioni di posizione e gradiente, scartando i vettori più vecchi non appena si supera la soglia prefissata.

### Algoritmo (L-BFGS)

Scegliere il punto iniziale  $x_0$ , un intero  $m > 0$ ;  
 $k \leftarrow 0$ ;  
repeat  
    Scegliere  $H_k^0$ ;  
    Calcolare  $p_k \leftarrow -H_k \nabla f_k$  dall'Algoritmo precedente;  
    Calcolare  $x_{k+1} \leftarrow x_k + \alpha_k p_k$ , dove  $\alpha_k$  è scelto in modo da soddisfare Wolfe;  
    if  $k > m$   
        Scartare la coppia di vettori  $\{s_{k-m}, y_{k-m}\}$  dalla memoria;  
    Calcolare e salvare  $s_k \leftarrow x_{k+1} - x_k$ ,  $y_k = \nabla f_{k+1} - \nabla f_k$ ;  
     $k \leftarrow k + 1$ ;  
until convergenza.

## 4 Ottimizzazione degli iperparametri

L'ottimizzazione degli iperparametri, nota in letteratura come *hyperparameter tuning*, rappresenta una fase cruciale nello sviluppo di un modello di classificazione.

Gli iperparametri sono configurazioni che devono essere definite a priori prima che l'addestramento abbia inizio, a differenza dei parametri interni del modello (come i pesi della regressione logistica) che vengono appresi dagli algoritmi di discesa nella fase di training.

In questo progetto, la ricerca si è concentrata nel trovare il valore ottimale dell'iperparametro di regolarizzazione  $\lambda$ . Valori di  $\lambda$  troppo piccoli rendono la regolarizzazione inefficace, esponendo il modello a un rischio di overfitting, mentre valori troppo elevati forzano i pesi ad approssimarsi allo zero, causando *underfitting* e rendendo il classificatore incapace di distinguere i pazienti sani da quelli malati.

### 4.1 K-Fold Stratificata e CV

Per prevenire l'overfitting e garantire una validità del classificatore, l'ottimizzazione degli iperparametri è stata condotta sul training set, costituito da 242 osservazioni, mentre il test set finale, composto da 61 pazienti è stato isolato e lasciato inalterato, in quanto esso valuta la capacità di generalizzazione del modello su dati mai visti prima.

La ricerca esplorativa dell'iperparametro  $\lambda$  è avvenuta tramite la tecnica della *Grid Search*, che consiste nell'esplorazione in una griglia di valori candidati: 0.001, 0.01, 0.1, 1, 10.

Sul training set è stata utilizzata la tecnica del *K-fold cross-validation*, attraverso l'utilizzo di  $K=5$  fold, che consiste nel dividere i dati di training in 5 sottoinsiemi di uguali dimensioni: iterativamente, 4 sottoinsiemi (con 193 pazienti) vengono utilizzati per il calcolo del gradiente e l'aggiornamento della posizione, mentre il quinto (49 pazienti) funge da set di validazione per misurarne le prestazioni.

In particolare, è stata utilizzata la variante stratificata (*Stratified*), in quanto in un contesto di diagnostica, per garantire di mantenere la stessa proporzione tra pazienti sani e malati all'interno di ognuno dei 5 fold. L'assenza di stratificazione avrebbe potuto generare fold sbilanciati, introducendo gravi bias di campionamento e compromettendo l'affidabilità del test.

## 4.2 Risultati della Grid Search

Come metrica decisiva per la selezione dell'iperparametro  $\lambda$  si è optato per la minimizzazione della *Log-Loss (Cross-Entropy)* calcolata sui fold di validazione, permette di misurare il reale potere di generalizzazione del modello e l'affidabilità delle sue probabilità cliniche.

Il valore di regolarizzazione ottimale per minimizzare la Log-Loss su dati non visti è pari a  $\lambda = 1.0$ .

Questa specifica configurazione è stata conseguentemente impiegata per il riaddestramento definitivo sull'intero training set.

## 5 Analisi della convergenza e confronto dei Metodi di Ottimizzazione

La Figura 2 mostra l'andamento della funzione loss sul training set in funzione del numero di iterazioni, rappresentate su scala logaritmica per facilitare il confronto visivo tra gli ordini di grandezza.

Dal grafico si nota come tutti i metodi convergano verso lo stesso minimo, attestandosi su un valore di loss pari a circa 0.35. Tuttavia, emergono differenze marcate in termini di velocità ed efficienza di convergenza:

- i metodi del secondo ordine (Newton e Newton-CG) mostrano una pendenza iniziale estremamente ripida, raggiungendo la convergenza rispettivamente in 6 e 7 iterazioni.
- I metodi Quasi-Newton si posizionano in una fascia intermedia l'algoritmo L-BFGS raggiunge il minimo in sole 37 iterazioni, ricalcando la pendenza iniziale dei metodi di Newton, mentre il BFGS standard ne richiede 75.
- il metodo del primo ordine di discesa del gradiente risulta nettamente il più lento, richiedendo ben 3310 iterazioni per raggiungere la convergenza.

L-BFGS rappresenta il miglior compromesso computazionale, raggiungendo una convergenza rapida senza la necessità di calcolare o invertire esplicitamente la

matrice Hessiana richiesta dal metodo di Newton puro.

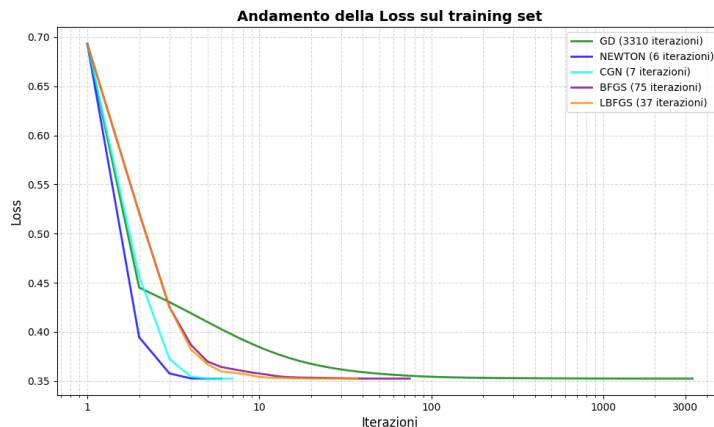


Figura 2: Andamento della funzione log-loss regolarizzata sul training set al variare del numero di iterazioni.

## 6 Valutazione del modello sul test set

Per valutare l'efficacia del modello di regressione logistica costruito, la si applica al test set, per misurare le reali prestazioni predittive. I risultati ottenuti vengono analizzati attraverso *metriche di valutazione*, in modo da quantificare la qualità delle predizioni del classificatore.

Nel caso di problemi di classificazione è indispensabile riassumere le prestazioni attraverso la *matrice di confusione*, un metodo di visualizzazione dei risultati che confronta le etichette predette con quelle reali presenti nel dataset. Quando ci si occupa di una classificazione binaria, i risultati si dividono in quattro categorie: Veri Positivi (TP), Veri Negativi (TN), Falsi Positivi (FP) e Falsi Negativi (FN).

La diagonale principale della matrice raccoglie i casi in cui il sistema classifica esattamente i dati, mentre gli altri valori indicano le predizioni errate.

|       |        |          |        |
|-------|--------|----------|--------|
| Reale | Sano   | 27       | 6      |
|       | Malato | 2        | 26     |
|       |        | Sano     | Malato |
|       |        | Predetto |        |

Figura 3: Matrice di confusione sul test set

La Matrice di confusione (Figura 3) valutata sul test set, composto da 61 pazienti, registra i seguenti valori:

- Veri Negativi (TN): 27 (pazienti sani correttamente classificati come sani);
- Falsi Positivi (FP): 6 (pazienti sani classificati erroneamente come affetti da patologia cardiaca);
- Falsi Negativi (FN): 2 (pazienti con patologia cardiaca ma erroneamente classificati dal modello);
- Veri Positivi (TP): 26 (pazienti con patologia cardiaca correttamente diagnosticati).

Questo comportamento denota un assetto conservativo del classificatore, considerato accettabile in scenari di screening medico dove un falso allarme, risolvibile con ulteriori accertamenti, è ampiamente preferibile a una mancata diagnosi.

A partire da questi valori, è possibile calcolare molte altre metriche. In particolare, sono state utilizzate:

- *Accuracy (Accuratezza)*: calcola la proporzione di predizioni corrette rispetto al totale, ovvero:

$$\text{Accuracy} = \frac{\text{Previsioni Corrette}}{\text{Totale Campioni}} = \frac{TP + TN}{TP + TN + FP + FN}.$$

Questa metrica è di semplice interpretazione, ma può essere fuorviante in presenza di dataset sbilanciati.

- *Recall (Sensibilità)*: indica quanti veri positivi sono stati identificati correttamente rispetto al totale dei positivi, ovvero:

$$\text{Recall} = \frac{TP}{TP + FN}.$$

Si ricerca un valore ottimale vicino a 1 per azzerare i falsi negativi, assicurando che nessun paziente sano venga erroneamente classificato come patologico.

- *Specificity (Specificità)*: indica quanti veri negativi sono stati identificati correttamente rispetto al totale dei negativi, ovvero:

$$\text{Specificity} = \frac{TN}{TN + FP}.$$

Si ricerca un valore ottimale vicino a 1 per azzerare i falsi positivi, assicurando che nessun paziente sano venga erroneamente classificata come patologica.

- *Precision (Precisione)*: detto anche Valore Predittivo Positivo (VPP), misura la qualità delle predizioni positive, indicando quanti dei campioni classificati come positivi lo siano effettivamente. Si calcola come:

$$\text{Precision} = \frac{TP}{TP + FP}.$$

Si ricerca un valore ottimale vicino a 1, che indica una riduzione al minimo dei falsi positivi.

- *Valore Predittivo Negativo (VPN)*: misura la qualità delle predizioni negative, indicando quanti dei campioni classificati come sani lo siano effettivamente. Si calcola come:

$$\text{VPN} = \frac{TN}{TN + FN}.$$

Si ricerca un valore ottimale vicino a 1 per garantire che un paziente diagnosticato come sano non sia in realtà malato.

- *F1-Score*: è dato dalla combinazione di Precision e Recall, calcolando la loro media armonica:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Produce un valore compreso tra 0 e 1: un valore vicino a 1 indica un equilibrio ideale tra precisione e richiamo, certificando la stabilità del classificatore anche in presenza di un dataset sbilanciato.

Le metriche di valutazione del modello sono riportate nella Tabella 2.

Dal punto di vista medico, l'errore più critico e pericoloso in un sistema di supporto alla diagnosi è rappresentato dai Falsi Negativi, in quanto comporta la mancata diagnosi e il rinvio a casa di un paziente malato senza cure.

Il modello proposto dimostra un comportamento ottimale per uno strumento di screening preventivo, raggiungendo un Recall (Sensibilità) del 93% e limitando i Falsi Negativi a soli 2 casi su 28 pazienti patologici. Di riflesso, l'elevata Precision sulla classe dei sani, che corrisponde al Valore Predittivo Negativo (93%) assicura che un responso negativo emesso dal sistema sia altamente affidabile. La Specificità, pur avendo un valore dell'82%, corrispondente a 6 Falsi Positivi, rappresenta un compromesso clinico ampiamente accettabile. In uno scenario reale, questi 6 pazienti sani verrebbero semplicemente sottoposti a una successiva valutazione o a esami più approfonditi, accettando un aumento dei controlli al fine di minimizzare i rischi clinici derivanti da una diagnosi errata.

| Classe / Metrica    | Precision | Recall | F1-Score    | Supporto |
|---------------------|-----------|--------|-------------|----------|
| Sani (0)            | 0.93      | 0.82   | 0.87        | 33       |
| Malati (1)          | 0.81      | 0.93   | 0.87        | 28       |
| <b>Accuracy</b>     |           |        | <b>0.87</b> | 61       |
| <b>Macro Avg</b>    | 0.87      | 0.87   | 0.87        | 61       |
| <b>Weighted Avg</b> | 0.88      | 0.87   | 0.87        | 61       |

Tabella 2: Metriche di classificazione dettagliate ottenute sul test set (61 pazienti)

## 6.1 Curva ROC e AUC

L'output di un modello di Regressione Logistica consiste in un valore continuo, calcolato tramite la funzione sigmoide, che quantifica la probabilità di appartenenza alla classe positiva (nel nostro caso, la presenza della patologia cardiaca). Per convertire questa probabilità in una classificazione binaria, il sistema applica una soglia decisionale, impostata a 0.5.

Per quantificare la stabilità e la robustezza del classificatore al variare di questo parametro, oltre alle metriche puntuali derivate dalla matrice di confusione, è necessario utilizzare metriche di valutazione indipendenti dalla singola soglia scelta.

A tal fine, si è scelto di analizzare la curva *ROC* (*Receiver Operating Characteristic*) e la relativa area sottesa *AUC* (*Area Under the Curve*).

La curva ROC si costruisce su un piano cartesiano dove l'asse delle ordinate rappresenta il tasso di veri positivi *TPR*, mentre l'asse delle ascisse indica il tasso di falsi positivi *FPR*. Questi due tassi si determinano rispettivamente come:

$$TPR = \frac{TP}{TP + FN} = \text{Recall} \quad \text{e} \quad FPR = \frac{FP}{FP + TN} = 1 - \text{Specificity}.$$

Il grafico viene generato modificando con continuità la soglia di classificazione all'interno dell'intervallo tra 0 e 1, calcolando i corrispondenti valori di *TPR* e *FPR* per ciascun punto.

Un modello le cui predizioni si basano sulla pura casualità produrrebbe una curva ROC approssimabile a una diagonale che congiunge l'origine (0,0) al punto (1,1).

Per valutare la curva così ottenuta, si utilizza l'*Area Under the Curve* (AUC). Questo valore numerico, compreso tra 0 e 1, equivale alla probabilità che il risultato del classificatore applicato a un dato estratto casualmente dal gruppo dei positivi sia superiore a quello ottenuto applicandolo a un dato estratto casualmente dal gruppo dei negativi.

Un valore di AUC pari a 0.5 indica un comportamento del tutto casuale (diagonale), mentre un valore pari a 1 dimostra una separazione perfetta delle ca-

tegorie, senza generare alcun errore di classificazione.

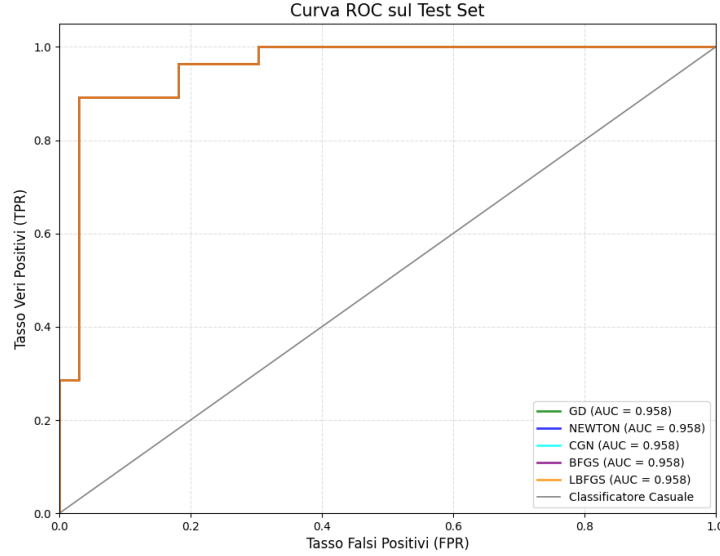


Figura 4: Curva ROC

L'analisi della Curva ROC, come mostrato in Figura 4, sul test set registra un valore di **AUC** pari a **0.958**, indipendentemente dal metodo di ottimizzazione impiegato. Questa sovrapposizione delle prestazioni predittive è diretta conseguenza di quanto osservato nell'analisi della loss: avendo tutti gli algoritmi raggiunto il medesimo minimo globale, essi hanno individuato lo stesso vettore dei pesi ottimali  $w^*$ .

La curva cresce in modo abbastanza rapido verso l'angolo superiore sinistro, il modello riesce a raggiungere un TPR alto mantenendo un FPR contenuto.

In termini pratici, selezionando casualmente un paziente con malattia cardiaca e uno sano, vi è una probabilità del 95.8% che il classificatore assegni un rischio maggiore al paziente effettivamente malato, confermando l'elevata affidabilità diagnostica del modello sviluppato.

## 6.2 Impatto delle feature e interpretabilità del classificatore

Analizzando il vettore dei pesi ottimali  $w^*$  appreso dal modello di Regressione logistica durante la fase di addestramento, è possibile quantificare l'impatto di ciascuna variabile sul processo decisionale.

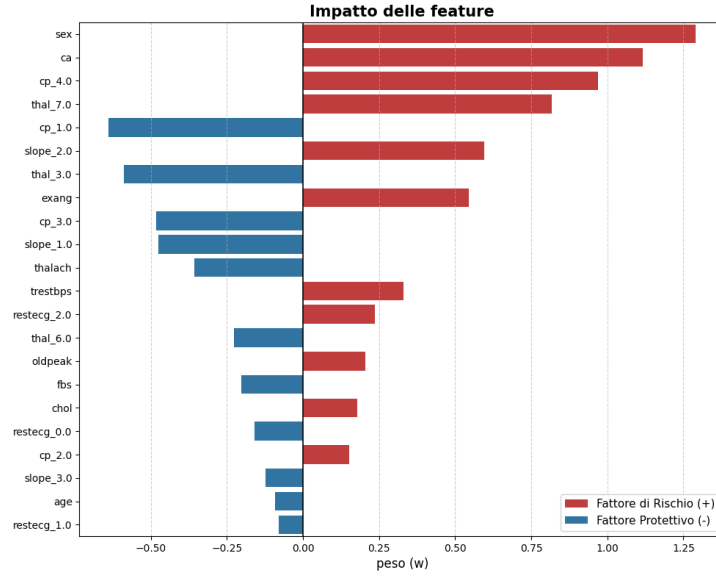


Figura 5: Impatto delle feature calcolato tramite i coefficienti della regressione logistica

Poiché le variabili continue sono state preventivamente standardizzate e le variabili categoriche convertite tramite *One-Hot Encoding*, i coefficienti calcolati sono direttamente confrontabili tra loro in termini di magnitudo (valore assoluto). Il segno del coefficiente determina se la variabile contribuisce positivamente o negativamente verso la predizione della classe.

Tra i principali fattori di rischio (associati alla classe positiva) emergono: la variabile *sex* ( $w = 1.2899$ ). A seguire, la variabile quantitativa *ca* (numero di vasi principali colorati dalla fluoroscopia) mostra un peso fortemente positivo ( $w = 1.1159$ ). La presenza di vasi sanguigni già ostruiti costituisce un danno in corso, giustificando l'elevato peso assegnato. Tra le variabili categoriche espanse, spiccano per rischio la presenza di dolore toracico asintomatico (*cp\_4.0*,  $w = 0.9682$ ) e la presenza di un difetto reversibile nel test della talassemia (*thal\_7.0*,  $w = 0.8162$ ) e l'angina indotta dall'esercizio fisico (*exang*,  $w = 0.5439$ ).

Tra i fattori protettivi, che invece orientano il classificatore verso la classe negativa (assenza di patologia), sono principalmente legate a risultati considerati "normali" negli esami e di buona tolleranza allo sforzo. La presenza di dolore toracico di tipo angina tipica (*cp\_1.0*,  $w = -0.6377$ ) e l'assenza di difetti alla talassemia (*thal\_3.0*,  $w = -0.5887$ ) agiscono come i più forti indicatori di salute. Inoltre, la frequenza cardiaca massima raggiunta (*thalach*,  $w = -0.3562$ ) si conferma un importante fattore protettivo: valori più elevati (che indicano una migliore tolleranza allo sforzo) si associano a probabilità inferiori di patologia coronarica.

I predittori meno influenti sono: l'età (*age*,  $w = -0.0921$ ) e il colesterolo (*chol*,  $w = 0.1781$ ), i quali mostrano pesi modesti e prossimi allo zero. Sebbene nella pratica medica costituiscano fattori di rischio in generale, in questo specifico contesto potrebbero risultare ininfluenti a causa della presenza di esami clinici e strumentali molto più diretti.

## 7 Conclusioni

In questo studio è stato implementato da zero un modello di regressione logistica, addestrato e validato per un task di classificazione di patologie cardiache.

Dal punto di vista teorico e computazionale, lo sviluppo di metodi di ottimizzazione unconstrained (Discesa del Gradiente, Metodo di Newton, Newton-CG, BFGS e L-BFGS) ha permesso confrontare la convergenza e l'efficienza dei diversi algoritmi. L'implementazione delle strategie di line search, basate sulle condizioni di Armijo e di Wolfe, ha garantito la stabilità dell'addestramento.

Tutti i solutori implementati hanno raggiunto lo stesso minimo della funzione obiettivo, evidenziando tuttavia come il costo computazionale e il numero di iterazioni richiesti per raggiungere la soluzione siano stati profondamente differenti tra i vari metodi. Il metodo L-BFGS si è confermato l'approccio più bilanciato ed efficiente in termini computazionali, rappresentando un compromesso ideale: da un lato supera nettamente la lentezza della convergenza lineare tipica del metodo del Gradiente Discendente (che ha richiesto 3310 iterazioni), dall'altro evita il costo computazionale e spaziale richiesto dall'allocazione e dall'inversione della matrice Hessiana nei metodi esatti del secondo ordine.

Il modello, dal punto di vista predittivo, ha ottenuto ottimi risultati sul test set, raggiungendo un'accuratezza dell' 87% e un valore di AUC pari a 0.958, che si allineano ai benchmark di riferimento presenti in letteratura. Inoltre, il modello mantiene un perfetto equilibrio, registrando un F1-Score dell'87% per entrambe le categorie. Il dato clinicamente più rilevante è l'elevata sensibilità (Recall) del 93% raggiunta nell'identificazione dei pazienti malati. Questo valore risulta cruciale in ambito diagnostico poiché riduce al minimo i falsi negativi, limitando il grave rischio di mancate diagnosi.

Infine, l'estrazione del vettore dei pesi ottimali  $w^*$  ha garantito la piena interpretabilità del classificatore, permettendo di identificare i fattori clinici più determinanti nello sviluppo della patologia, come il numero di vasi ostruiti alla fluoroscopia, la presenza di dolore toracico asintomatico, e il difetto reversibile alla scintigrafia con tallo.

## Riferimenti bibliografici

- [1] Andras Janosi et al. *Heart Disease*. UCI Machine Learning Repository. 1989. DOI: [10.24432/C52P4X](https://doi.org/10.24432/C52P4X).
- [2] Anderies Anderies et al. «Prediction of Heart Disease UCI Dataset Using Machine Learning Algorithms». In: *Engineering, Mathematics and Computer Science (EMACS) Journal* 4 (set. 2022), pp. 87–93. DOI: [10.21512/emacsjournal.v4i3.8683](https://doi.org/10.21512/emacsjournal.v4i3.8683).
- [3] Jorge Nocedal e Stephen J. Wright. *Numerical Optimization*. 2<sup>a</sup> ed. Springer, 2006.
- [4] Andrew Ng e Tengyu Ma. *CS229 Lecture Notes*. [https://cs229.stanford.edu/main\\_notes.pdf](https://cs229.stanford.edu/main_notes.pdf). Giu. 2023.
- [5] David G. Luenberger e Yinyu Ye. *Linear and Nonlinear Programming*. 4<sup>a</sup> ed. Vol. 228. International Series in Operations Research & Management Science. Springer, 2016. ISBN: 978-3-319-18841-6. DOI: [10.1007/978-3-319-18842-3](https://doi.org/10.1007/978-3-319-18842-3).
- [6] Dimitri P. Bertsekas. *Nonlinear Programming*. 3<sup>a</sup> ed. Belmont, Massachusetts: Athena Scientific, 2016. ISBN: 978-1-886529-05-2.
- [7] Chiradeep Gupta et al. «Cardiac Disease Prediction using Supervised Machine Learning Techniques». In: *Journal of Physics: Conference Series* 2161.1 (2022). DOI: [10.1088/1742-6596/2161/1/012013](https://doi.org/10.1088/1742-6596/2161/1/012013).
- [8] Stephen Boyd e Lieven Vandenberghe. *Convex Optimization*. Cambridge: Cambridge University Press, 2004. ISBN: 978-0-521-83378-3. DOI: [10.1017/CB09780511804441](https://doi.org/10.1017/CB09780511804441).
- [9] Shreyas Raman et al. «Machine Learning for Coronary Heart Disease Prediction: Comparative Analysis of Framingham and Cleveland Subset of the UCI Dataset with SHAP-Based Interpretability». In: *Epidemiologia* 7.3 (2026). ISSN: 2673-3986. DOI: [10.3390/epidemiologia7030075](https://doi.org/10.3390/epidemiologia7030075). URL: <https://www.mdpi.com/2673-3986/7/3/75>.
- [10] Ambrish G et al. «Logistic regression technique for prediction of cardiovascular disease». In: *Global Transitions Proceedings* 3.1 (2022). International Conference on Intelligent Engineering Approach(ICIEA-2022). ISSN: 2666-285X. DOI: <https://doi.org/10.1016/j.gltp.2022.04.008>. URL: <https://www.sciencedirect.com/science/article/pii/S2666285X22000449>.
- [11] Jamal Al-Karaki et al. *Predicting Coronary Heart Disease Using a Suite of Machine Learning Models*. 2024. arXiv: [2409.14231](https://arxiv.org/abs/2409.14231) [cs.AI]. URL: <https://arxiv.org/abs/2409.14231>.
- [12] M. D. Teja e G. M. Rayalu. «Optimizing heart disease diagnosis with advanced machine learning models: a comparison of predictive performance». In: *BMC Cardiovascular Disorders* 25.1 (2025), p. 212. DOI: [10.1186/s12872-025-04627-6](https://doi.org/10.1186/s12872-025-04627-6).